

Trading Interference for Retrieval Difficulty:

A Content-Addressed Continual Memory with On-Demand Capacity

Xinyu Yang

pana.yang@hotmail.com

ORCID: 0009-0007-2600-0948

August 29, 2026

Abstract

A learner with bounded parameters and an unbounded stream must give something up: regularisation gives up plasticity, training on regardless gives up the past. A third option is to *allocate*, giving distinct regimes distinct storage, and the standing objection is that adding parameters per task defers the problem rather than solving it. We take that objection seriously by building the most committed version of allocation we could — storage addressed by content through a fixed random map, so no later learning can move where a fact lives; regimes inferred online with no task identifier, boundary or replay; and capacity appended rather than resized, so adding storage leaves existing weights and predictions bit-identical — and asking what it buys.

Our reading is that allocation does not remove interference so much as *convert* it, trading destruction of old material for difficulty in retrieving among more of it. We argue this is the better failure, because retrieval difficulty is non-destructive and answerable by addressing better while interference is answerable only by spending parameters; we report the exchange rate we measured and the mitigations it implies. The same commitment has a price: an address fixed by content is precisely what keeps context out of retrieval.

On a synthetic relational stream the mechanism retains substantially more than a monolithic control where regimes are distinguishable, allocates capacity in proportion to emitted content rather than vocabulary size, and holds per-token cost independent of context length. Other readings came out flat and we report them as open: six measurements in this study that read as results proved to have been produced by the measurement rather than the mechanism, and a synthetic source cannot exhibit what it was not built to contain.

1 Introduction

A learner that must absorb an unbounded stream with bounded parameters has to discard something. Regularisation methods such as elastic weight consolidation [Kirkpatrick et al., 2017] give up plasticity, protecting old weights at the cost of new learning. Methods that simply keep training give up the past. A third option is to *allocate*: give distinct regimes distinct storage, so that degradation is uniform rather than selective. Dynamically expanding architectures [Rusu et al., 2016, Yoon et al., 2018] take this route and are routinely criticised for adding parameters per task without demonstrating that the addition is proportionate to new content rather than to task count.

This paper examines an allocation mechanism built around three commitments. Addressing is by content through a fixed random hash, so a fact reaches the same storage under every history. Regime identity is inferred online from the stream, so no task identifier, task boundary, or replay buffer is required. Capacity is appended rather than resized, so adding storage leaves what is already there untouched, which we check exactly rather than statistically.

We report what these commitments buy and what they cost. They buy a large advantage where regimes are distinguishable, a capacity footprint that follows emitted content rather than

vocabulary size, and a per-token cost independent of sequence length. They cost the ability to associate items across a gap, which content-only addressing forbids by construction, and they leave retention declining as regimes accumulate.

Contributions.

1. An allocation mechanism whose two addressing factors — content to a memory slice, inferred regime to a readout — are separately measurable, with no task supervision at any point (§3).
2. Evidence that sharing the payload transform while privatising the readout is the effective split for a stream whose regimes share surface form and differ in targets, confirmed from both directions (§5.2).
3. A capacity result: sampling negatives from a regime’s own emitted targets makes occupancy follow content, $7.6\times$ smaller at equal accuracy (§5.3).
4. A non-destructiveness property, verified exactly rather than statistically: appending capacity leaves stored weights and predictions unchanged bit for bit (§5.4).
5. Measurements of what the mechanism does not currently do — cross-gap association, multi-hop addressing, and retention as regimes accumulate — reported with the alternative explanations we could exclude, and left open where we could not (§6).

2 Related work

Continual learning. Catastrophic interference in distributed representations [McCloskey and Cohen, 1989, Ratcliff, 1990, French, 1999] motivates three broad families [Parisi et al., 2019, De Lange et al., 2021]. Regularisation slows change on weights estimated to matter [Kirkpatrick et al., 2017, Zenke et al., 2017, Li and Hoiem, 2017]; rehearsal replays or distills stored exemplars [Rebuffi et al., 2017, Lopez-Paz and Ranzato, 2017, Chaudhry et al., 2019]; architectural expansion gives new material new parameters [Rusu et al., 2016, Yoon et al., 2018]. Our mechanism is architectural but requires no task boundary to decide when to expand: the decision comes from the dispersion of an inferred regime’s own statistics, which places it in the task-free setting [Aljundi et al., 2019] and, in the taxonomy of van de Ven and Tolias [2019], in the hardest scenario, where neither a task label nor a task boundary is supplied at training or at test time. The idea that separating storage is a solution rather than an evasion is older than the deep-learning literature on it: complementary learning systems argue that a fast, sparse, pattern-separating store is what makes slow cortical consolidation survivable [McClelland et al., 1995, Kumaran et al., 2016].

Conditional computation and routing. Structurally the mechanism is a single-layer, top-1 sparsely-gated mixture of experts [Jacobs et al., 1991, Bengio et al., 2013, Shazeer et al., 2017, Fedus et al., 2022] whose expert count grows. Its router is the unusual part. Learned gates need load-balancing losses and can collapse; the alternatives replace learning with a rule — a balanced assignment problem [Lewis et al., 2021], or a fixed hash of the token [Roller et al., 2021], which is the closest prior work to what we do and reports that a fixed hash is competitive with a learned gate. We differ in what the fixed rule buys and what it costs. Hashing there is a way to avoid the pathologies of a learned gate; here it is a *stability* guarantee, because an address that cannot move is an address no subsequent learning can invalidate. The cost is the same in both cases and we report it as a limitation rather than a feature: no load balancing, no learned specialisation, and — specific to a continual setting — no way for context to influence retrieval (§6).

Addressable and distributed memory. Content-addressed storage with sparse codes follows sparse distributed memory [Kanerva, 1988] and hyperdimensional computing [Kanerva, 2009], whose capacity arguments descend from associative networks [Hopfield, 1982] and, in their modern attention-equivalent form, from Ramsauer et al. [2021]. Differentiable external memories [Weston et al., 2015, Graves et al., 2016] learn both what to store and where; we learn only the former, which is what makes the address stable and also what forbids the mechanism from learning a better one. Recoverable association of *pairs*, which a smoothed context summary cannot provide, is available through holographic reduced representations [Plate, 1995] or fast-weight outer products [Schmidhuber, 1992, Ba et al., 2016, Schlag et al., 2021]; we identify this as the missing ingredient rather than claim it.

Non-parametric and retrieval memory. An alternative to storing knowledge in weights is to retrieve it at inference from an external store, either by nearest neighbours over cached representations [Khandelwal et al., 2020] or by retrieving text [Guu et al., 2020, Lewis et al., 2020, Borgeaud et al., 2022]. This shares our premise that new knowledge should not require gradient updates to a shared representation, and differs in where the burden falls: a retrieval corpus grows in storage and search cost but needs no allocation policy, whereas we pay an allocation decision and a growing candidate set in exchange for a store that is written online, in one pass, and is itself the model. The retrieval-difficulty scaling we report (§6) is the analogue of the index-size problem in that literature.

Consolidation across timescales. Cascades of variables at multiple timescales [Fusi et al., 2005, Benna and Fusi, 2016] target the capacity–lifetime tradeoff of bounded synapses. We tested such a cascade in two roles, as weight consolidation and as input context, and report both outcomes.

Attention and its costs. Self-attention [Vaswani et al., 2017] pays $O(N)$ per token in context access and degrades on material in the middle of long contexts [Liu et al., 2024]. A large literature reduces that cost by sparsifying or linearising the attention pattern [Beltagy et al., 2020, Katharopoulos et al., 2020, Tay et al., 2022], and recent state-space models achieve constant per-token cost with a recurrent state [Gu and Dao, 2023]. Our mechanism sits with the latter in cost profile and differs in what the constant-size state is for: a state-space model compresses the sequence into a vector that the network learns to read, while here the constant-size summary is used only to *address*, and what is retrieved comes from a store that grows with content rather than from the summary itself. We quantify the crossover in §5.5.

Vocabulary growth and tokenisation. That distinct types grow sublinearly in a Zipf-distributed stream [Zipf, 1949, Piantadosi, 2014] is Heaps’ law [Heaps, 1978]. We use it to ask whether allocated capacity tracks content, and find that our generator does not reproduce the sublinear regime, which makes it an unfavourable source for the question (§5.3). Subword vocabularies [Sennrich et al., 2016] are constructed to flatten exactly the frequency structure this mechanism reads, and are known to be a suboptimal segmentation for pretraining on other grounds as well [Bostrom and Durrett, 2020]; we return to this in §8.

Evaluation protocol. We evaluate prequentially — predict, take the loss, then write [Dawid, 1984] — which is standard for stream learning [Gama et al., 2013] and has the property that no example is ever scored after it has been trained on [Blier and Ollivier, 2018]. In a single-pass setting this is the only protocol under which a retention number means what it appears to mean.

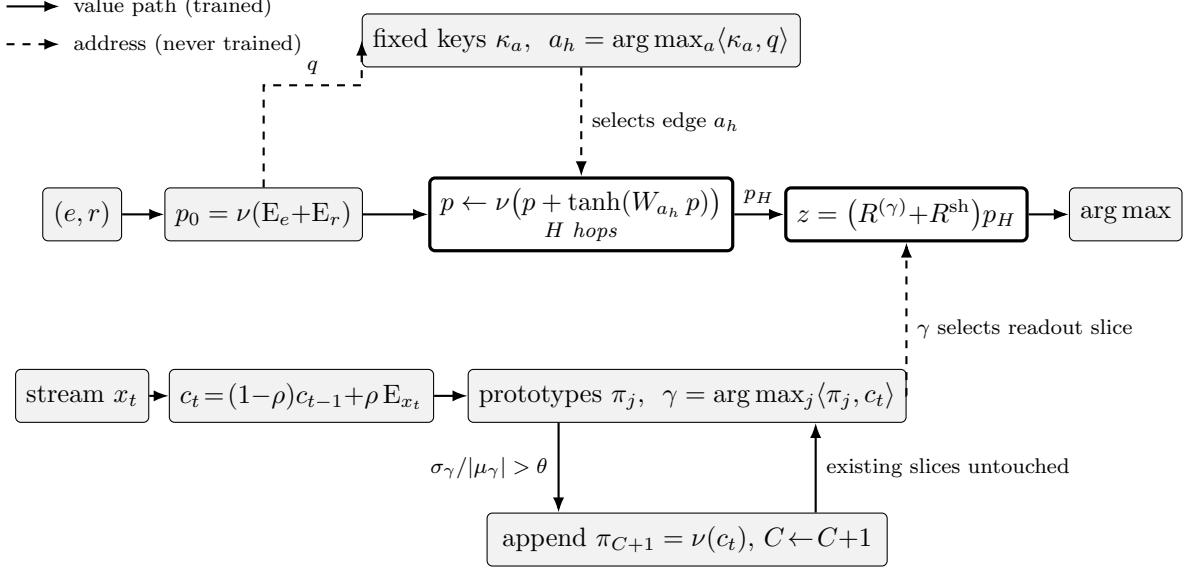


Figure 1: The mechanism. Two addressing factors, neither of them learned, decide *where* an observation is stored; the two heavy-outlined blocks are the only ones holding trained parameters. Content selects the edge, so a fact reaches the same transform under every history (§3.2); the regime inferred from the stream selects the readout slice, so facts that disagree across regimes never share a row while the transform they pass through stays common. Capacity is appended when a class’s own similarity distribution disperses, which by §3.6 leaves every existing offset and value untouched. The routing key q equals p_0 in every configuration reported here; R^{sh} holds the rows more than one class has emitted.

3 Method

3.1 Notation

Let $\mathcal{V}$ be the vocabulary, $V = |\mathcal{V}|$, and d the code width. The embedding table $E \in \mathbb{R}^{V \times d}$ has rows drawn independently and uniformly from the unit sphere S^{d-1} at construction and is *never* updated; E_x denotes the row of token x . Write $\nu(x) = x/\|x\|_2$ for ℓ_2 normalisation and δ_t for the indicator vector of token t .

Storage lives on a small-world ring $G = (\mathcal{N}, \mathcal{E})$ with $|\mathcal{N}| = n$ nodes: node u has out-neighbours consisting of its two ring neighbours plus a fixed number of random shortcuts, giving out-degree D_u and $|\mathcal{E}| = M = \sum_u D_u$ directed edges. Every edge $a \in \mathcal{E}$ carries two objects: a *key* $\kappa_a \in \mathbb{R}^d$, drawn once at construction and never updated, and a learned matrix $W_a \in \mathbb{R}^{d \times d}$ shared across regimes. Each class c owns a readout $R^{(c)} \in \mathbb{R}^{V \times d}$ and a prototype $\pi_c \in S^{d-1}$. The number of live classes C is not fixed in advance. It starts at one — the mechanism is not told how many regimes exist — and grows from the stream (§3.5).

A fact is a triple $(e, r, t) \in \mathcal{V}^3$: entity, relation, target. The stream is a sequence of tokens $x_1, x_2, \dots$ from which facts are read off, seen once each in the order they arrive.

3.2 Two addressing factors

Content to edge. The payload and the address are computed from the same tokens but kept separate, so that the transform and readout always act on content:

$$p_0 = \nu(E_e + E_r), \quad q = \nu\left(p_0 + \frac{g}{\sqrt{m}} P \xi_t\right), \quad (1)$$

where $\xi_t \in \mathbb{R}^m$ is a concatenation of context traces at several timescales, P a fixed random projection and g a gain. The main configuration uses $g = 0$, in which case $q = p_0$ and the

address depends on the fact alone; $g > 0$ is the variant of §6. Entry and each hop are argmaxes over fixed keys,

$$u_0 = \arg \max_{u \in \mathcal{N}} \langle \kappa_{(u,1)}, q \rangle, \quad a_h = \arg \max_{a \in \text{out}(u_h)} \langle \kappa_a, q \rangle, \quad u_{h+1} = \text{head}(a_h), \quad (2)$$

for $h = 0, \dots, H-1$. Since κ is fixed and, at $g = 0$, q is a deterministic function of (e, r) , the walked path $(a_0, \dots, a_{H-1})$ is a deterministic function of the fact. Two consequences follow without measurement: routing consistency — the fraction of probed facts that return to the edges their writes went to — is exactly 1, and no amount of subsequent learning can move a stored fact’s address. We report 1.000 in the tables as a check on the implementation, not as a result.

Regime to readout. A slow exponential average of the stream, with rate ρ , is matched against the prototypes:

$$c_t = c_{t-1} + \rho (E_{x_t} - c_{t-1}), \quad \gamma_t = \arg \max_{j \leq C} \langle \pi_j, c_t \rangle. \quad (3)$$

No task identifier enters anywhere: γ is inferred from the stream, and the mechanism is never told which regime is live, nor when one ends. The prototypes receive no gradient; they are *placed* (§3.5), not trained.

3.3 Memory and forward pass

The value path applies the selected edge transform residually,

$$p_{h+1} = \nu(p_h + \tanh(W_{a_h} p_h)), \quad h = 0, \dots, H-1, \quad (4)$$

with $\tanh$ elementwise. Rows are owned exclusively: a target emitted by more than one class is held in a block R^{sh} common to all of them, every other target in the live class’s own block. Scoring reads both,

$$z = (R^{(\gamma)} + R^{\text{sh}}) p_H \in \mathbb{R}^V, \quad \hat{y} = \text{softmax}(z), \quad (5)$$

while a write goes to exactly one of the two, so no target is stored twice.

The edge transform is shared across regimes and the readout is not. The split follows the generative structure rather than preference: a payload is built from an entity and a relation, which are common across regimes, whereas targets are exactly what regimes disagree about. Section 5.2 confirms this from both directions — against a fully private variant, and by testing the opposite split.

3.4 Learning

Training is online, single-pass and prequential [Dawid, 1984, Gama et al., 2013]: for each fact the model predicts, is charged the loss, and only then writes. There is no replay buffer, no task boundary, no epoch and no second pass.

The readout takes a delta-rule step restricted to a *touched set*

$$T = \{t\} \cup S, \quad S \sim \text{Unif}^k(\mathcal{T}_\gamma), \quad R_v^{(\gamma)} += \eta(\mathbb{I}[v=t] - \hat{y}_v) p_H^\top \quad \text{for } v \in T, \quad (6)$$

where $\mathcal{T}_\gamma$ is the set of targets class γ has already emitted and k the number of sampled negatives, drawn with replacement by a deterministic counter-based generator and with the target itself discarded ($k = 0$ recovers the dense update over all V rows). Rows outside T are left *exactly* zero rather than nearly zero, which is what makes occupied capacity a measure of stored content (§5.3).

The error is then backpropagated along the path that was actually walked, and along no other. With $\nabla_{p_H} L = (R^{(\gamma)})^\top (\hat{y} - \delta_t)$, each hop is differentiated exactly through the normalisation and the residual tanh block of (4): writing $b_h = W_{a_h} p_h$, $u_h = p_h + \tanh(b_h)$ and $\lambda_h = \|u_h\|$,

$$\nabla_{u_h} L = \frac{1}{\lambda_h} \left(I - p_{h+1} p_{h+1}^\top \right) \nabla_{p_{h+1}} L, \quad \nabla_{b_h} L = (1 - \tanh^2(b_h)) \odot \nabla_{u_h} L, \quad (7)$$

$$W_{a_h} \leftarrow \eta \nabla_{b_h} L p_h^\top, \quad \nabla_{p_h} L = \nabla_{u_h} L + (W_{a_h})^\top \nabla_{b_h} L. \quad (8)$$

The gradient reaches neither the keys κ , nor the prototypes π , nor the embedding table: every quantity that decides *where* an observation goes is outside the computational graph.

Two implementation points affect how the equations above should be read. The weight update of (8) is applied in place before $\nabla_{p_h} L$ is formed, so the backward signal passes through the post-update matrix rather than the one used in the forward pass. And the mechanism supports a decay term, applied lazily by carrying a per-slice scale and materialising a slice only when it is touched — algebraically exact rather than an approximation — but it is set to zero throughout the experiments reported here, so nothing decays in any measurement below.

3.5 Allocation

Let $\text{sim}_t = \max_j \langle \pi_j, c_t \rangle$ and let $\mu_\gamma, \sigma_\gamma^2$ be running estimates of its mean and variance conditioned on the class γ that won. Once both the global and the per-class estimates have accumulated a minimum number of observations, a class is split when its own similarity distribution is too dispersed for τ consecutive steps,

$$\frac{\sigma_\gamma}{\max(|\mu_\gamma|, \epsilon)} > \theta, \quad \text{whereupon} \quad \pi_{C+1} \leftarrow \nu(c_t), \quad C \leftarrow C + 1. \quad (9)$$

Dispersion is used rather than an outlier test because outlier tests fail in two ways we measured. Against a *global* running distribution of sim they fail once regimes are numerous, since that distribution becomes their mixture and nothing is unusual relative to it. Against a class's *own* distribution an outlier test fails too, because a class absorbing more regimes raises its own variance, so a threshold of the form $\mu_\gamma - \theta \sigma_\gamma$ falls exactly when splitting becomes more necessary. Dispersion is scale-free in both senses: a class serving one regime has tightly clustered similarities whatever the value of C .

The new prototype is placed at the current context, and allocation appends, leaving existing slices at their offsets and values.

3.6 Expansion is exact

The readout is stored class-major, so class c occupies a contiguous block at offset $c \cdot Vd$, and the shared transform carries no class index at all. Appending class $C + 1$ therefore changes no offset of any class $j \leq C$ and writes no byte inside one. Consequently, for any input whose inferred regime satisfies $\gamma \leq C$, the logits $z = R^{(\gamma)} p_H$ are *bit*-identical before and after expansion, since the arithmetic performed is the same on the same operands in the same order. This is a property of the layout, and we assert it as a unit test on weights and logits rather than estimating it statistically.

Two distinct things are called growth here and should be kept apart. Allocation raises the live count C within a block of slices reserved at construction, which is what happens in every run reported below. Appending *beyond* that reservation is a separate code path, enabled by a flag that is off in these runs, and it is the one the exactness argument and the unit test concern. The reservation is a ceiling on how far allocation can go, not a statement about how much capacity is used: what the occupancy figures of §5.3 measure is how much of it is written to.

3.7 Cost

Per observed token, addressing costs $O((n + H\bar{D})d)$ for the argmaxes of (2) and $O(Cd)$ for (3); the memory costs $O(Hd^2)$ for (4); the readout costs $O(Vd)$ to score and $O(|T|d)$ to write. None of these depends on how many tokens preceded, which is the property of interest: dot-product attention over a context of N tokens costs $O(Nd)$ per token for the same function, and the two cross at $N = d$. The readout term is the one that does not yet follow occupancy, and it is what §8 proposes to make sparse.

4 Experimental setup

The source is a multi-domain relational graph walked continuously, with facts stratified by visit frequency into hub, mid and tail tiers under a Zipf distribution. Two modes are used. In **Mode A** each regime owns its entities, so regime identity is present in the input. In **Mode B** entities and relations are byte-identical across regimes and only targets differ, which removes the routing signal and, by construction, any shared target to amortise; we use it as an adversarial bound rather than as the main setting.

Evaluation is zero-shot retention of probed facts under a frozen memory, with the probe restoring the context it needs and returning the state it displaced. All figures in this paper are extracted by a single parser that anchors on the results table header and reads the column by name.

	symbol	flag	value
source	—	--mode	a
	—	--domains	8
	V	--vocab	16384
	—	--facts-per-domain	150
	—	--span-tokens / --rounds	4000 / 6
	s	--zipf-s / --hub-ratio	1.0 / 0.10
	—	--target-zipf	on
control	—	--d-sdr / --k-active	512 / 16
	—	--d-input / --m-in	32 / 8
mechanism	n	--edge-nodes	16
	—	--edge-shortcuts	2 (default)
	d	--edge-dim	32
	H	--edge-hops	varied per experiment
	—	--edge-classes / --edge-init-classes	64 / 1
	θ, τ	--edge-grow-k / --edge-grow-hold	0.05 / 2
	—	--edge-grow-orth	off
	—	--edge-share / --edge-share-readout	on / on
	k	--edge-neg-samples	32
off	g	--edge-ctx-gain, --edge-forget	0, 0
	—	--edge-posterior, --edge-hash-class,	
	—	--edge-gate, --edge-clip, --edge-expand	off / 0

Table 1: The configuration of the routing comparison, given in full as a reference point; each result elsewhere names the parameters it varies. The learning rate is $\eta = 1.0$ and the elastic-weight baseline is disabled. Four entries decide how the equations of §3 should be read: --edge-ctx-gain = 0 makes $q = p_0$, so the address is a function of the fact alone; --edge-forget = 0 means nothing decays; --edge-grow-orth off means a new prototype is placed at the context rather than on its orthogonal residual; and --edge-share makes the transform common to all classes, so the edge memory carries no class index.

5 Results

5.1 Where regimes are distinguishable

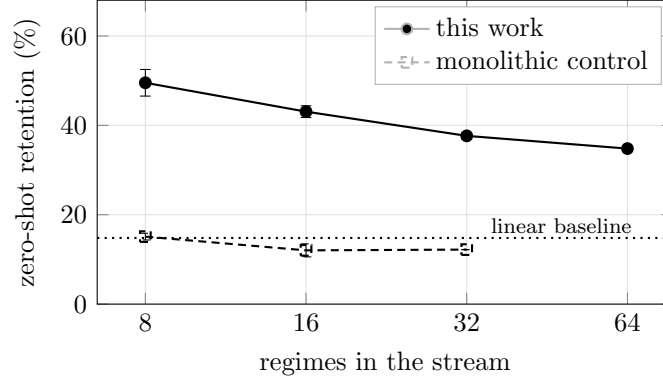


Figure 2: Mode A. Bars are one standard deviation across seeds ($n = 3$ at eight and sixteen, $n = 2$ at thirty-two and sixty-four for this work; $n = 3, 3, 1$ for the control). The control sits at the level of a linear baseline without sparse coding, so the task is not solvable by predicting frequent targets; the dotted line is that baseline at eight regimes, 14.80% over the same three seeds. The sixty-four-regime control was not obtained: a single cell costs about nine hours here and the run was lost.

Where a regime is identifiable from its own inputs, allocation retains 49.53%, 43.07% and 37.65% of probed facts at eight, sixteen and thirty-two regimes, against 15.10%, 12.03% and 12.20% for a monolithic sparse-coded control of the same width: advantages of +34.43, +31.04 and +25.45 points (Figure 2). The advantage narrows as regimes accumulate but does not close over the range we could measure. Both series decline, and §6 asks why ours does.

5.2 Sharing the transform, privatising the readout

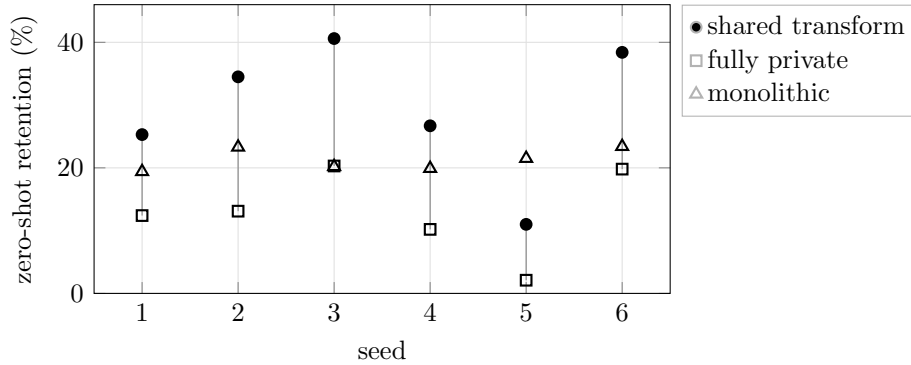


Figure 3: Mode B, twelve regimes, per seed. Absolute retention varies by a factor of nearly four across seeds (11.0 to 40.6), which is why the paired comparison is the one that carries weight: the vertical connectors are all the same sign, +8.9 to +21.4 points, 6/6. Against the monolithic control the same comparison is 5/6 and not established.

Sharing the *readout* as well costs 27.50 points on this source, and here the mechanism of the loss is legible rather than inferred. A common readout block is updated on every write and therefore accumulates every regime’s target mapping, while Mode B is defined by the same (e, r) pointing at different targets per regime; it averages contradictions. The asymmetry of §3 is thus confirmed from both sides, one by making the change and the other by testing its opposite.

Mode B, twelve regimes, six seeds	zero-shot	paired difference
Shared transform, private readouts	29.42%	+16.43pp, 6/6, $p < 0.001$
Fully private	12.98%	—
Monolithic	21.28%	+8.13pp, 5/6, $p = 0.119$

Table 2: Sharing the transform is established against the private architecture. The advantage over the monolithic control is not.

5.3 Capacity follows content

	occupied rows	rows per class	accuracy
Dense delta rule	45056/45056 (100%)	4096	32.6%
Sampled from emitted targets	5918/45056 (13.1%)	538	33.8%

Table 3: A $7.6\times$ reduction in occupied capacity at no cost in accuracy.

One qualification belongs with this result rather than in the limitations. The quantity capacity is supposed to follow is the union of distinct emitted targets, and Heaps’ law predicts that union to grow sublinearly in a Zipf-distributed stream [Heaps, 1978]. Our generator does not produce that: the union measured at eight, sixteen, thirty-two and sixty-four regimes is 273, 534, 1066 and 2111 distinct targets, or 34.2, 33.4, 33.3 and 33.0 per regime — flat, so the union grows *linearly* in regimes. Each regime samples its own entity and target pool, so there is little shared vocabulary to amortise, and the sublinear growth that makes allocation attractive on natural text is absent by construction. The reading this permits is the narrow one: occupancy tracks emitted content rather than vocabulary size. Whether that translates into sublinear capacity growth requires a source with Heaps’ law structure, which ours is not.

5.4 Expansion does not disturb what is stored

Appending a class slice leaves every existing weight byte-identical and **forward** bit-identical, asserted as a test rather than measured statistically. A monolithic model cannot offer this: its capacity is the sparse-coding width, and widening it changes the projection through which every stored code was written.

5.5 Cost per token is independent of context length

Memory access costs Hd^2 per token regardless of sequence length, against Nd per token for dot-product attention over a context of N tokens; the crossover is at $N = d$ and the ratio is $128\times$ at $N = 4096$ (Figure 4). Against ourselves the reading is less flattering. In the runs behind §5.1, activation is 1,049,600 of 269,547,520 memory parameters per token, or 0.389%; but of that activation the edge memory accounts for 1,024 and the readout, scoring all $V = 32768$ rows, for 1,048,576. The expert half is sparse and the readout half is not, and the half that is not is a thousand times the larger. What is constant in context length is the total; what sets that constant is the dense half.

6 Open questions

Every negative reading in this study that we examined closely turned out, at least once, to be produced by the measurement rather than by the mechanism (Appendix A). We therefore report the following as measurements whose interpretation is unsettled, rather than as properties of

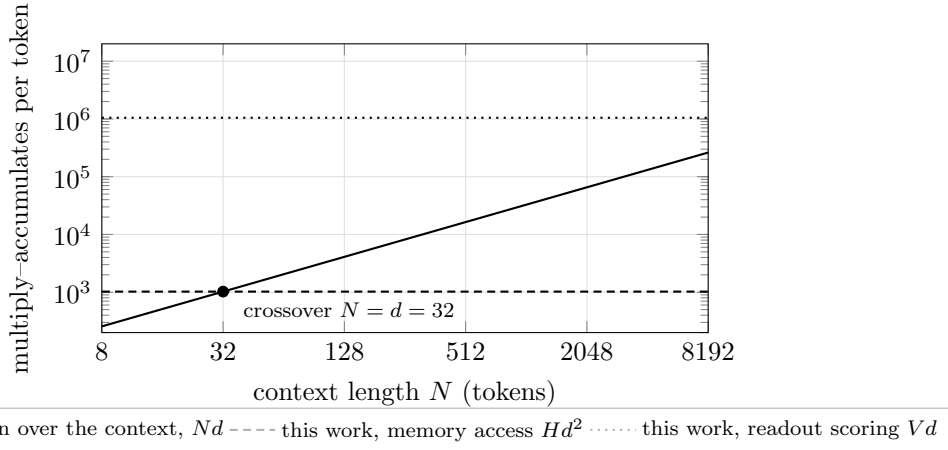


Figure 4: Per-token cost at the configuration behind §5.1 ($d = 32$, $H = 1$, $V = 32768$). The memory access is flat in N and crosses attention at $N = d$; at $N = 4096$ the ratio is $128\times$. The qualification is the dotted line: the readout scores all V rows, so it exceeds the memory access by a factor of 1024 and dominates the total. It too is flat in N — the cost is constant in context length, but the constant is set by the dense half. Making it follow occupancy instead is the first variant of §8: a class holds 39–41 occupied rows of the 32768 scored.

the architecture. Three qualifications apply throughout and we state them once here rather than repeating them. The source is synthetic, so an effect the generator cannot express is an effect we cannot observe. Seed counts are small enough that a null result is compatible with a modest true effect. And in each case there are changes we have not tried that the diagnosis itself suggests; where we can name them, we do.

Can the mechanism associate across a gap? Nothing is carried between facts beyond a single-timescale context, so a cue survives $(1 - \rho)^{\text{gap}}$.

gap	cue remaining	accuracy
1	0.990	13.23%
16	0.851	13.97%
64	0.526	13.50%
256	0.076	10.87%

Table 4: Three seeds. Cue remaining is $(1 - \rho)^{\text{gap}}$ at the context rate used, and its collapse between gap 64 and gap 256 was predicted from that rate before the run. Paired degradation from gap 1 to gap 256 is $+2.37\text{pp}$, $2/3$ seeds, $t(2) = 1.55$, $p \approx 0.26$: on this source and at this seed count we did not establish the degradation, only its direction. An earlier batch of three seeds gave $+3.27\text{pp}$, $5/6$, $p = 0.013$ when pooled with these, but it ran before the evaluation was changed to charge a model for carrying nothing, and the two batches differ by a factor of two in level, so we report the post-change batch alone rather than the pooled six.

Exact association under constant memory is information-theoretically unavailable, so the design question is what a lossy code should retain. A multi-scale average answers *which regime* and cannot answer *which item*. Placing context into the address recovers roughly a tenth of the gap and costs lookup accuracy, consistent with content-only addressing being what forbids association in the first place.

The evaluation itself is unfavourable to the mechanism, and we want that on the record. Each run fixes a single gap, and a single gap is best served by whichever single time constant

happens to match it; a multi-scale context can therefore show no advantage under this protocol by construction, because the comparison is decided by the evaluation rather than by the code.

Under a mixed-gap protocol that cycles horizons within a run, the sign of the effect changes: the strongest context gain we tried is worth +0.67pp on average against no context in the address (44.47% $\rightarrow$ 45.13%, two seeds of three in the favourable direction), where under a single fixed gap the same setting bought +0.40pp at the long end while costing 0.47pp at the short one. We report this as a sign change and nothing more, and three facts about it should be read alongside. The response is not monotone in the amount of context — the intermediate gain gave 43.00%, below the no-context arm — which is consistent either with a genuine non-monotonicity or with the differences being noise. The spread across seeds, 37.0 to 52.4 per cent, is an order of magnitude larger than the effect. And three seeds cannot establish a difference of this size in either direction. What the sign change is consistent with is the protocol rather than the code having decided the earlier comparison; separating those two would need the gain swept more finely, more seeds at each setting, and a source whose dependency horizons are not fixed by construction. We regard both protocols as preliminary: a gap constructed in a synthetic stream fixes what has to be carried and for how long, and cannot show what a real dependency structure demands. Establishing this needs evaluation on natural language at scale, which we have not done.

Whether this is a limit of the mechanism or of this particular probe is not settled. Three earlier versions of the same measurement were invalid, each for a different reason, and the fourth is the one reported here; placing context into the address recovers roughly a tenth of the gap, which suggests the addressing scheme rather than the context code may be the binding constraint.

hops	seeds	zero-shot	occupied rows
1	2	55.15%	750
2	1	46.00%	737
3	3	29.63%	744

Table 5: Occupancy is the same to within 2% while retention falls by half. The seed counts are unequal and small, and the two-hop cell is a single run; on the two seeds all three depths share, the ordering is the same (54.2, 56.1 at one hop against 31.2, 25.8 at three). Under-training is excluded as an explanation: writes per edge *rise* with depth, averaging 3152, 6432 and 8965 at one, two and three hops over roughly the same 24–26 live edges, because each fact touches more of them.

Does multi-hop addressing ever pay? We do not conclude that depth is unnecessary; the setup is narrow enough that we could not, and we set out why before reporting what we saw. Multi-hop is the only form of depth this architecture has, and it is depth in the transform rather than in the routing: hop $h + 1$ transforms hop h ’s output, but every hop selects its edge with the same fixed copy of p_0 , so the sequence of edges a fact visits is settled by its original content. An earlier version of this comparison was taken while regime separation had collapsed; the present one uses a single source, one scale, and a lookup task whose targets are determined entirely by the current fact. A source that rewards composition — where an answer follows from chaining two stored relations rather than retrieving one — is exactly where combinatorial addressing would be expected to pay, and we have not built one. What the table shows is that on a source requiring no path, adding one did not pay for itself. We would expect that to change on a source where an answer follows from chaining two stored relations, and three routes are open to testing it: a generator with genuine two-step dependencies; an address that composes rather than repeats, which we tried in one form (§8) and could try in others; and separating

depth in the transform from depth in the routing, so that the two can be varied independently. None of these is expensive, and until one is run the reading here is about this source rather than about depth.

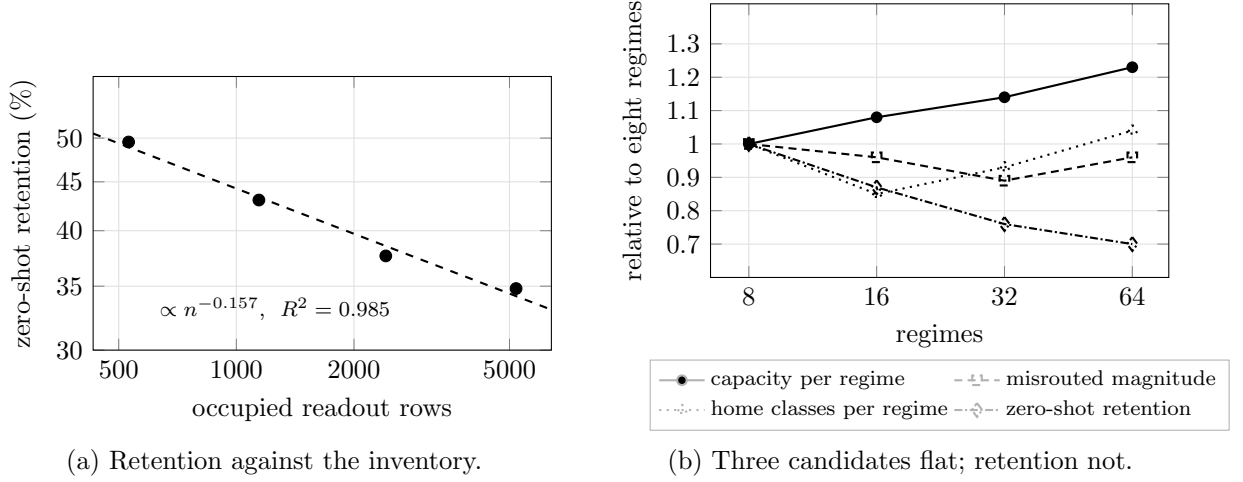


Figure 5: Three explanations for the decline are excluded, leaving one. Both panels are normalised to the eight-regime value. **(b)** Capacity per regime *rises* ($66 \rightarrow 81$ occupied rows per regime), so this is not starvation; misrouted write magnitude is flat (57.0, 54.7, 51.0, 55.0 per cent), so it is not worsening interference; home classes per regime is flat (0.75, 0.64, 0.70, 0.78), so separation does not degrade. Retention is the only one of the four that moves. **(a)** The quantity it moves with is the inventory the argmax must discriminate among, which grows $9.8\times$ over the same range; against that inventory the decline is close to a power law, $\propto n^{-0.157}$ with $R^2 = 0.985$ on four points. Note that the prediction distribution does not collapse: distinct targets predicted rise from 47 to 57 as accuracy falls, which is what distinguishes a discrimination failure from a representational one. Four points and two to three seeds per point are too few to fix an exponent; we read the figure as ruling three accounts out, not as measuring a law.

Why does retention decline as regimes accumulate? Three explanations are excluded by the data (Figure 5). Capacity is supplied faster than linearly in regimes, so this is not starvation. Misrouted write magnitude is flat, so it is not worsening interference. Home classes per regime is flat, so separation does not degrade. What remains is retrieval: the inventory of occupied readout rows — the set the argmax must discriminate among — grows $9.8\times$ while accuracy falls to $0.70\times$, and distinct predictions rise from 47 to 57 rather than collapsing.

These exclusions constrain the explanation without establishing it. We state the retrieval account as the surviving candidate rather than as the finding, and note what would settle it: the account predicts that bounding the scored set to a class’s occupied rows, or partitioning the inventory so that no single candidate set grows without limit, should move the exponent, while adding capacity should not. Both are cheap to run and neither has been run.

Does directed forgetting reclaim anything? We report no number. Every measurement we made of it is confounded, so what follows is an account of what went wrong rather than a result. The earlier ones ran while a per-domain snapshot keyed by ground truth was reaching the training path, and the later one, which added capacity pressure so that a freed slice would be worth something, ran while the allocation criterion was collapsing every regime into a single class — so there were no separate slices to reclaim in the first place. We report no number for it, and note only that a valid test requires contention between regimes for storage *and* a working allocator, which we did not have simultaneously.

Does acquiring a regime become cheaper as regimes accumulate? Later regimes do cost less per unit accuracy in our runs, but our measurement staggers arrival, and arrival order is itself sufficient to produce the effect: a regime arriving late enters a system whose transform and prototypes are already shaped. Separating order from content requires a design we have not run, so we leave the question open rather than answer it from a measurement that cannot distinguish the two (Appendix B).

Is the decline with scale a power law? Over the measured range, retention falls as $n^{-\alpha}$ with $\alpha = 0.157$ and $R^2 = 0.985$, n being the number of occupied readout rows (Figure 5a). Four points and two to three seeds per point cannot distinguish a power law from any other slowly decaying form, so we report the fit as a compact summary of the range measured and not as a law. Whether the exponent is a property of the mechanism, and whether stronger addressing rather than more capacity moves it, is measurable without new machinery: the hierarchical variant of §8 predicts it should fall.

What should a constant-memory context retain? Exact association across a gap is information-theoretically unavailable under constant memory, so the design question is what to keep. A multi-scale average answers *which regime* and cannot answer *which item*. Sparse retention of salient items, or superposition with approximate unbinding [Plate, 1995, Ba et al., 2016], keeps identity where an average cannot.

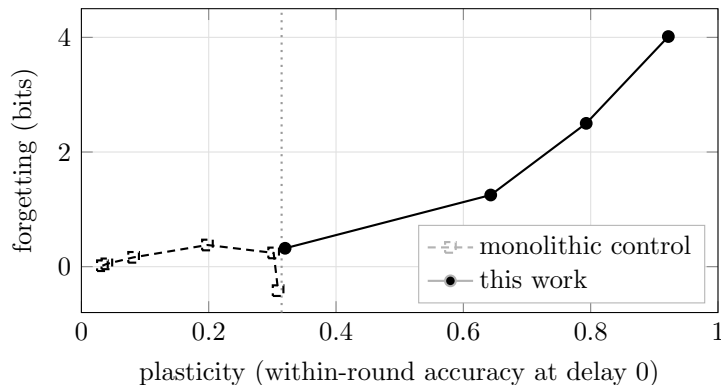


Figure 6: Each point is one learning rate, two or three seeds, Mode B at twelve regimes; the rate sweeps $\eta \in \{0.1, \dots, 30\}$ for the control and $\{0.1, \dots, 3\}$ for ours. The two arms do not overlap in plasticity at all: the control saturates at 0.309 and ours starts at 0.320, either side of the dotted line. Nothing about frontier *position* can be read from disjoint supports, which is why we report reachability only. Two features are worth stating rather than smoothing over. Our forgetting grows steeply across our own range, 0.32 to 4.01 bits, so the high-plasticity end is not free — though the learning rate is the only knob swept here, and allocation thresholds, negative sampling and the readout ladder all move the same tradeoff and were held fixed. And the control’s highest-rate cell has *negative* forgetting, which we do not have an account for; we read it as the control becoming unstable at that rate rather than as a stability result, and it is the cell that most needs more seeds.

Does the plasticity–stability frontier extend or shift? The frontier traced by our mechanism and that traced by the monolithic control do not overlap in plasticity at all (Figure 6), so at present only reachability can be compared, not position. Making the comparison possible needs a regime in which both arms operate at the same plasticity, and we did not construct one.

7 Discussion

7.1 Why exact context cannot be cheap

To associate any point in a stream with any other exactly, a system must be able to answer for an arbitrary pair, which requires either retaining every state and comparing against them — $O(N)$ memory and $O(N)$ work per query — or precomputing the pairs, $O(N^2)$ in total. Self-attention pays exactly this and buys exactness within its window in return [Vaswani et al., 2017]. The cost is not an implementation artefact; it is what pairwise access costs.

Under constant memory the question changes. Once the stream is longer than what the memory can hold distinctly, some pairs must become unanswerable, and the design decision is which ones. This is the same shape of decision that continual learning already forces on the weights: bounded capacity against an unbounded stream means something is given up, and methods differ in what [Kirkpatrick et al., 2017, Rusu et al., 2016]. Our mechanism extends that choice to the context.

7.2 A fuzzy background instead of a transcript

The position we take, and the one we find natural given the biological case, is that a learner does not need a transcript of its context. A slowly varying state that says *what kind of situation this is* can serve as an anchor for retrieval, leaving the identity of individual past items unrecoverable. Our context is exactly that: an exponential summary matched against prototypes, which conditions retrieval without recording what occurred.

Our measurements track this division cleanly. The mechanism answers “which regime” well — regimes separate, and retention exceeds the monolithic control by 25 to 34 points — and fails at “which item occurred” as soon as a cue must survive a gap (§6). That is the anchor working as an anchor and failing as a record, which is what it is.

We do not claim this is the only division, or the best one. At least four alternatives are available and differ in what they keep: sparse retention of a few salient items keeps identity and discards time; superposition with approximate unbinding keeps many items at graceful degradation [Plate, 1995]; outer-product fast weights keep pairs at a capacity that scales with d^2 [Ba et al., 2016, Schlag et al., 2021]; and a hybrid in which a fuzzy anchor selects a slice while a small exact buffer covers the recent past would give up long-range exactness only. Which of these is right is an empirical question we have not settled, and the answer may well depend on what the stream is.

7.3 Where this leaves three familiar problems

Catastrophic forgetting. Allocation does not solve interference so much as convert it. Where a shared representation loses old material as new material overwrites it [McCloskey and Cohen, 1989], storage that is separated by regime removes that channel and substitutes another: retrieval among a growing inventory. The exchange rate looks favourable in our measurements — confusable stored targets grow $9.8\times$ while retention falls to $0.70\times$, and the distinct predictions made rise rather than collapse — but it is a substitution, not a removal, and the cost reappears as scale.

The stronger reason to prefer it is not the rate but the shape of the two failures. Interference in a shared representation is destructive and open-ended: material that is overwritten is gone, and the only remedies are more capacity or less learning. Retrieval difficulty is non-destructive and sub-proportional: the material is still stored, merely harder to find, and a tenfold larger inventory cost us thirty per cent of retention. More importantly the two differ in what can be done about them. Interference is answered only by spending parameters; retrieval difficulty is answered by addressing better, which is a design axis rather than a budget.

That said, an exponent of roughly 0.16 compounds, and a mechanism whose retention decays with everything it has ever learned would not scale. Several remedies follow directly from the diagnosis, and we list them in order of how directly they attack the measured quantity, which is the number of candidates a retrieval must discriminate among.

1. *Bound the scored set.* Inference currently scores all V rows of the live slice while a class holds 39–41 occupied rows. Scoring only occupied rows makes the candidate count follow content directly, which is the quantity in the exponent (§8).
2. *Shard the memory.* A coarse router over several independent memories, each with its own allocator, keeps any one inventory bounded while total capacity grows with the number of shards — a mixture of experts whose experts are memories rather than layers [Shazeer et al., 2017, Fedus et al., 2022], routed by a rule rather than a learned gate as in Roller et al. [2021]. Growth then adds shards rather than enlarging a single candidate set, so the exponent becomes a property of shard size instead of a property of everything ever learned. The cost is a second routing decision that can be wrong, and the open question is whether material that belongs to two shards must be duplicated.
3. *Address hierarchically.* Within a shard, a coarse code selecting a slice and a fine code over that slice’s own targets makes discrimination logarithmic in the inventory rather than linear, at the cost of a coarse-level error being unrecoverable at the fine level (§8).
4. *Grow the code with the inventory.* Discriminating N items needs a code whose capacity keeps pace; letting d grow like $\log N$ would hold the exponent at the price of memory per row and of the byte-identical expansion property, since widening the code changes existing rows.
5. *Merge redundant slices.* If allocation over-splits, inventory reflects duplicates rather than distinct content. Consolidating slices whose similarity profiles coincide would keep the inventory a measure of what is actually distinct.
6. *Sample negatives adversarially.* Discrimination is currently trained against negatives drawn from a regime’s own emitted targets. Drawing them from the current nearest confusables instead spends the code where collisions actually occur.

The first two are the ones we would try first, and the second is the one that changes the asymptote rather than the constant.

Lost in the middle. Attention degrades on material by its position in the context [Liu et al., 2024]. Content addressing has no position term at all: a fact stored early is reached by the same hash as one stored late, so retrieval carries no positional decay. What decays instead is regime identification, because the context summary is recency-weighted, and what degrades with load is retrieval, because the inventory grows. The degradation is therefore moved off position and onto inventory and recency, which is a different failure surface rather than a smaller one — a prediction that is testable directly, and that we have not tested.

Plasticity and stability. On this source we cannot place the two frontiers relative to one another, and the reason is a property of the comparison rather than a result about either arm. Our mechanism and the monolithic control barely overlap in plasticity: the control saturates near 0.31 while ours spans 0.32 to 0.92, so at present the honest statement is about reachability, not position. Whether the frontier is extended or merely shifted requires a regime where both operate, which we did not construct.

7.4 What this is not for, and what it might be for

A statically trained model, evaluated as a static tool, remains the accuracy ceiling. It is trained to convergence over a fixed corpus with many passes; a single-pass online memory will not match it on a benchmark drawn from that same corpus, and we do not suggest otherwise. For answering ordinary questions from knowledge fixed at training time, the static model is the right instrument and this one is not.

The case for a mechanism like ours is different in kind: acceptable accuracy combined with the ability to keep working. We would also note that accuracy on a fixed source is a narrow proxy. It measures what a model knows at the moment it was frozen, not what it can absorb afterwards, not what it costs to keep running, and not what happens after weeks of operation — none of which a static benchmark is constructed to reveal. The properties measured here point at settings where those are the binding constraints.

Sessions measured in weeks. Per-token cost does not grow with context and there is no cache to retain, so an agent can run for as long as the task lasts rather than as long as a window allows. What it absorbed on the first day is stored rather than scrolled out.

Embodied and IoT settings. A robot or sensor node meets regimes that were not enumerated in advance and receives them once, in order, without labels. Allocation from the stream and single-pass writing match that shape; retraining cycles do not.

Edge and green deployment. Context access is Hd^2 per token independently of history, and a write touches one slice and a handful of rows. The cost profile is close to flat in the length of the deployment, which is the quantity that dominates energy over a long-running installation.

Absorbing new domains after deployment. Capacity appends without disturbing what is stored, so a fielded model can take on a new domain without a retraining cycle. The exactness matters more than the magnitude: one can state that adding capacity did not change existing behaviour, rather than measure that it changed little.

Targeted deletion, and interpretability more broadly. A regime occupies identifiable slices, so removing one is a local operation rather than a retraining problem — the natural test of the same structure that makes expansion non-destructive, and one we have not run. The same property supports weaker but broader questions: which slice serves this material, what did this write change, what would removing this regime cost. A learned router gives no comparable account of itself, and a dense model gives none at all. We think this is the more interesting direction of the two, since deletion is one query among many that an addressable, inspectable store can answer.

Long-tail material. Capacity follows emitted content rather than vocabulary size, so rare items occupy their own rows instead of being averaged into frequent neighbours.

7.5 Fabrication

We did not measure hallucination and make no claim about rates. But three structural properties bear on it and are worth stating, since each is a consequence of the architecture rather than of tuning.

Training and inference are not separate modes: the same forward path writes and reads, and what a query returns is what some slice holds. There is no generation stage distinct from retrieval in which content can be composed that was never stored.

Regimes occupy separate readout slices, so an answer is produced from the slice the context selected. Blending material across domains — one plausible route to a confident wrong answer — is limited by the same separation that limits interference, rather than by a penalty term.

Sparse occupancy makes absence representable. A class holds tens of occupied rows out of tens of thousands, so “nothing here scores well” is a state the mechanism can be in, which a dense readout normalised over the full vocabulary cannot easily express. Whether this translates into calibrated abstention is untested and would be the obvious experiment.

7.6 On whether growing capacity is disqualifying

Expansion invites the objection that a method which adds parameters has not solved anything. We think the objection is right about unbounded growth and wrong as a general principle, and the biological case is instructive on the distinction. A brain does not hold its knowledge in a fixed parameter vector: connectivity is combinatorial, signalling is graded rather than discrete, and the number of distinguishable synaptic states is correspondingly large. A system with that much effective capacity can afford to allocate in proportion to content, because content is small relative to what it can hold.

Digital parameters are finite and discrete, so the same strategy is only tenable if per-shard inventory is bounded — which is the argument for sharding above, and the reason we regard unbounded growth in a single memory as a defect to be fixed rather than a property to be defended. What the biological case does suggest is that allocation proportional to content is not disqualifying in itself. What must be bounded is not the total, but the number of things any single retrieval has to discriminate among.

8 Architectural variants worth trying

Each variant below targets a specific measurement in this paper rather than a general intuition.

Bound scoring to the emitted rows. Inference scores all V rows of the live readout slice, while a class holds 39–41 occupied rows against a vocabulary of 32768. Restricting the argmax to rows the class has emitted, with the remainder contributing a constant, would cut inference cost by nearly three orders of magnitude and would make the activation figure of §5.5 sparse in both halves rather than only in the expert half. Nothing about the mechanism resists this; it is unimplemented, not difficult.

Address hierarchically to bound the candidate set. The decline with scale tracks the number of confusable stored items rather than capacity, misrouting or separation (§6). If that account is right, the fix is not more capacity but a smaller candidate set at retrieval: a coarse regime code selecting a slice, then a within-slice code over that regime’s own targets, so that discrimination is against tens of alternatives rather than thousands. This predicts that the exponent α should fall with the depth of the addressing hierarchy, which is directly testable.

Add a term that stores pairs. A smoothed context can report which regime is active and cannot report which item occurred, so no amount of extra timescales will support association across a gap. An outer-product term, $M \leftarrow \lambda M + p_t \otimes p_{t-\Delta}$ retrieved as Mp , stores pairs recoverably at a capacity that scales with d^2 [Ba et al., 2016, Schlag et al., 2021], and is the same algebraic object as the edge transform already present. The open question is whether it can sit alongside content addressing without dissolving the stability that content addressing provides.

Give the router a learned residual. The fixed random hash buys routing consistency of 1.000 and forgoes load balancing and learned specialisation. A learned correction added to the key, small enough that a fact’s destination remains stable within a regime, would recover part of what a gated mixture of experts gets from its router [Shazeer et al., 2017] without giving up the property that makes expansion non-destructive.

Let the address compose, not only the value. The value path already composes: hop $h + 1$ transforms hop h ’s output and gradients flow along the walked chain. What repeats is the *address* — every hop selects its edge with the same fixed copy of p_0 , so the sequence of edges a fact visits is settled before the walk begins.

We implemented the obvious remedy, copying the transformed payload into the routing key after each hop so that hop $h + 1$ selects from hop h 's output, and we did not see it pay in the form we implemented. Within its own configuration, which uses a larger ring than the multi-hop table above and is therefore not directly comparable to it, accuracy falls from 51.43% to 41.03% at two hops and from 35.10% to 22.83% at three, so the gap to a single hop widens rather than closes. The reason is visible in the same run. Routing consistency, which the fixed key guarantees at 1.000, falls to 0.875 at two hops and 0.740 at three: a fact's path now depends on weights that move, so an eighth and then a quarter of probed facts no longer reach where training left them. The loss tracks the depth, which is what one would expect if it compounds along the walk rather than being a fixed cost. This settles the version we tried and not the idea. On a lookup whose target is fixed by the current fact there is nothing for a composed path to find, so the source cannot show the benefit even if it exists; and the loss we do see is attributable to a specific, fixable defect rather than to composition itself. Three variants would separate the two: recomputing the address from a *detached* copy of the transformed payload, so the path stops depending on weights that move; committing the path at write time and replaying it at read time, which keeps composition while restoring exact consistency; and evaluating on a generator with genuine two-step dependencies. Only the last requires new data.

A note on tokenisation. We recommend against subword tokenisation for this architecture. Byte-pair encoding [Sennrich et al., 2016] is designed to flatten the frequency distribution, and this mechanism depends on that distribution in two places: allocation splits a class when its similarity dispersion rises, which requires regimes to differ in what they emit, and capacity follows the count of distinct emitted targets, which requires a target to be a unit rather than a sequence. A rare entity split into several frequent subwords loses its identity twice over — its payload becomes a sum of common pieces, which collides in the content hash with every other entity sharing those pieces, and its target becomes a sequence with no single row to occupy. The long tail is exactly where a continual learner has something to gain, and subword vocabularies are built to spend it. A word- or entity-level vocabulary, or any tokenisation that keeps rare units intact, preserves both the Zipf structure the allocator reads [Zipf, 1949, Piantadosi, 2014] and the one-target-one-row correspondence the capacity argument relies on. That subword segmentation is suboptimal has been argued on unrelated grounds [Bostrom and Durrett, 2020]; our objection is specific to a mechanism that allocates by emitted content.

9 Limitations

Results are on a synthetic relational source; no natural-language corpus is evaluated, and a stream we constructed fixes what has to be learned and what can be shared, which is precisely what a real corpus would decide for us.

The monolithic control is established at eight, sixteen and thirty-two regimes, with one seed at thirty-two and none at sixty-four; our own cells at thirty-two and sixty-four have two seeds each. The comparison is not parameter-matched in either direction. At eight regimes the control holds a 512×32768 readout, 16.8M weights, while our mechanism allocated 10, 11 and 13 class slices across the three seeds — 327,680 to 425,984 rows, 10.5M to 13.6M weights — of which 523 to 538 rows, about 17K weights, are occupied. These are two models of comparable width and different size, so the occupancy figures describe our mechanism and are not a like-for-like efficiency claim against the control.

Cost comparisons between the dense and sampled write paths are not commensurable, because write magnitude accumulates over a different number of rows in each. The private and shared arms likewise differ in parameter count, which strengthens the accuracy comparison between them but confounds any comparison of their costs.

Sample sizes are small throughout: six seeds for the sharing result, three for most others, one for the largest control. Several effects in this study weakened or reversed between three seeds and six, so single-digit seed counts should be read accordingly.

Whether acquiring a regime becomes cheaper as regimes accumulate is not settled here: our measurement staggers arrival, and arrival order alone can produce the effect (Appendix B).

10 Conclusion

Content addressing with on-demand allocation buys a large advantage where regimes are distinguishable, a capacity footprint that follows emitted content, and a per-token cost independent of context length, without disturbing what is already stored. The same commitment that makes storage stable — addressing by content alone — is what limits association across a gap, and the size of that trade is measured here rather than asserted. What we cannot yet say is how much of the rest is a property of the mechanism and how much of the synthetic source we built to test it. The measurements that came out flat came out flat on a generator with no long-range dependency structure, no compositional targets and no natural Zipf tail, evaluated over single-digit seed counts; each of them names, in its own section, a change to the source or to the mechanism that would decide the question. Running those on natural text at scale is the work we would do next, and until it is done the readings above are about this source and this scale.

A Measurements invalidated during this study

Six measurements were invalidated after collection and re-run. Three were probe-side leaks in which a per-domain snapshot keyed by ground truth handed the mechanism the regime identity it was meant to infer. One was the allocation criterion of §3.5 in an earlier form, which compared novelty against a global running distribution and collapsed every regime into a single class at scale, voiding the savings, capacity and multi-hop measurements taken under it. One was a parser that scanned past the results table into a table with the same column count and reported 54.2% as 100.00%.

Each was caught by a contradiction between statistics rather than by reading code: a count that disagreed with another count, a diagnostic that printed nothing, an accuracy that could not coexist with its own exposure figures. The instruments that survived — write locality by time since regime change, occupancy against emitted content, allocation timing against regime boundaries — are reported alongside the results because they are what made the failures visible.

B Arrival order and the cost of a late regime

Measuring whether a later regime is cheaper to acquire requires staggered arrival, and staggering introduces a confound we record here for the benefit of anyone running the same measurement.

	shared transform	private
Mode A, staggered arrival	0.754	0.820
Mode B, disjoint targets, staggered arrival	0.442	0.505
Mode A, no staggering (index only)	1.199	1.207

Table 6: Cost of the late regimes relative to the early ones; below one means a late regime is cheaper. Three seeds per control.

Mode B’s targets are disjoint across regimes by construction, so nothing there can be amortised, yet the ratio is lower than in Mode A. Removing the staggering removes the effect. A regime arriving late therefore benefits from entering an already-shaped system, independently of whether it shares content with its predecessors, and a ratio below one cannot on its own be read as amortisation.

References

- I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*, 2013.
- L. Blier and Y. Ollivier. The description length of deep learning models. In *NeurIPS*, 2018.
- S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *ICML*, 2022.
- K. Bostrom and G. Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of EMNLP*, 2020.
- A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato. On tiny episodic memories in continual learning. *arXiv:1902.10486*, 2019.
- A. P. Dawid. Present position and potential developments: Some personal views. Statistical theory: The prequential approach. *Journal of the Royal Statistical Society A*, 147(2):278–292, 1984.
- J. Gama, R. Sebastião, and P. P. Rodrigues. On evaluating stream learning algorithms. *Machine Learning*, 90(3):317–346, 2013.
- A. Graves, G. Wayne, M. Reynolds, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016.
- A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. REALM: Retrieval-augmented language model pre-training. In *ICML*, 2020.
- J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 79(8):2554–2558, 1982.
- R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.
- U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through memorization: Nearest neighbor language models. In *ICLR*, 2020.
- D. Kumaran, D. Hassabis, and J. L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

- P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- M. Lewis, S. Bhosale, T. Dettmers, N. Goyal, and L. Zettlemoyer. BASE layers: Simplifying training of large, sparse models. In *ICML*, 2021.
- Z. Li and D. Hoiem. Learning without forgetting. *IEEE TPAMI*, 40(12):2935–2947, 2017.
- D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017.
- J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- S. T. Piantadosi. Zipf’s word frequency law in natural language: A critical review and future directions. *Psychonomic Bulletin & Review*, 21(5):1112–1130, 2014.
- H. Ramsauer, B. Schöfl, J. Lehner, et al. Hopfield networks is all you need. In *ICLR*, 2021.
- R. Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2):285–308, 1990.
- S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. iCaRL: Incremental classifier and representation learning. In *CVPR*, 2017.
- S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In *NeurIPS*, 2021.
- J. Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.
- Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.
- G. M. van de Ven and A. S. Tolias. Three scenarios for continual learning. *arXiv:1904.07734*, 2019.
- J. Weston, S. Chopra, and A. Bordes. Memory networks. In *ICLR*, 2015.
- F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017.
- G. K. Zipf. *Human Behavior and the Principle of Least Effort*. Addison-Wesley, 1949.
- R. Aljundi, K. Kelchtermans, and T. Tuytelaars. Task-free continual learning. In *CVPR*, 2019.
- J. Ba, G. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu. Using fast weights to attend to the recent past. In *NeurIPS*, 2016.
- M. K. Benna and S. Fusi. Computational principles of synaptic memory consolidation. *Nature Neuroscience*, 2016.
- M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE TPAMI*, 2021.

- W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.
- R. M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 1999.
- S. Fusi, P. J. Drew, and L. F. Abbott. Cascade models of synaptically stored memories. *Neuron*, 2005.
- H. S. Heaps. *Information Retrieval: Computational and Theoretical Aspects*. Academic Press, 1978.
- P. Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- P. Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 2009.
- J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 2017.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *TACL*, 2024.
- M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 1989.
- G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 2019.
- T. A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 1995.
- A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv:1606.04671*, 2016.
- I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- J. Yoon, E. Yang, J. Lee, and S. J. Hwang. Lifelong learning with dynamically expandable networks. In *ICLR*, 2018.